EE5311 Digital Circuits and Simulation
Tutorial 10: Registered Adder Trees with OpenLane

Venkata Subhash M - NS25Z205

Assignment Questions

EE5311 Digital IC Design: Assignment 10
Adder trees - Maximum operating frequency

1. Add a clock input and register the existing inputs/outputs at the posedge of clock for the three 16-bit tree adders in Assignment 9.
2. Use `OpenLane` to generate layouts and obtain post-layout STA report for the three adders.
3. Find the maximum operating frequency of the input clock for each tree adder.

Figure 1: Tutorial 10 Assignment Questions

Section 1: Registered Ripple Carry Adder

This section covers the registered version of the 16-bit Ripple Carry Adder implementation with input and output registers for synchronous operation.

1.1 Ripple Carry Adder with Registers

The registered ripple carry adder includes flip-flops at the input and output stages to enable synchronous operation and facilitate accurate timing analysis.

Ripple Carry Adder Verilog code

```
module adder (output reg[16:0] sum, input[15:0] a, input[15:0] b, input
    logic[15:0] a_q, b_q;
    logic cin_q;
    logic[16:0] sum_d;
    assign sum_d = {1'b0, a_q} + {1'b0, b_q} + {16'b0, cin_q};
    always @ (posedge clk) begin
        a_q <= a;
        b_q <= b;
        cin_q <= cin;
        sum <= sum_d;
    end
endmodule
```

OpenLane Layout

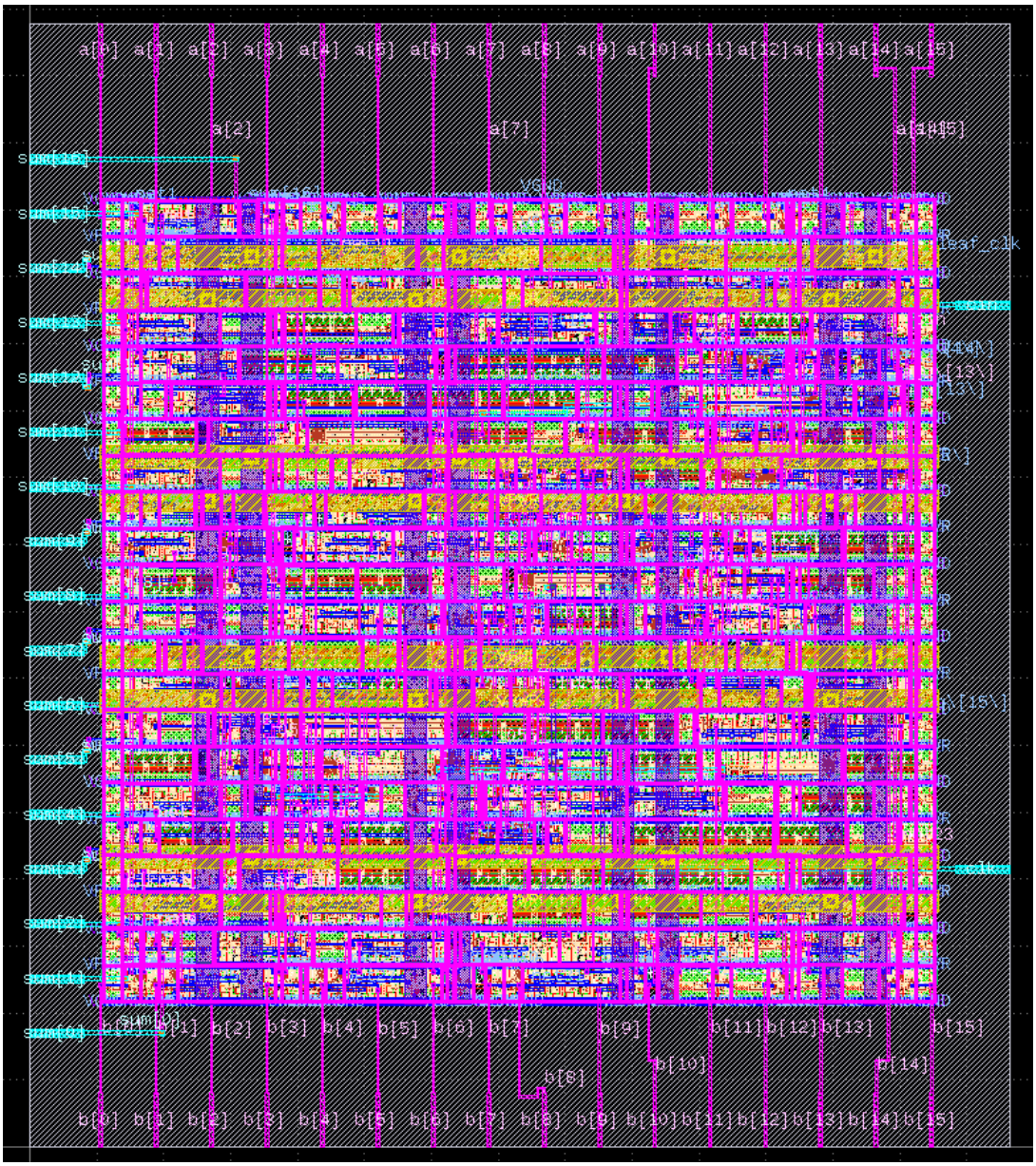

Figure 2: OpenLane Layout for Registered Ripple Carry Adder

Static Timing Analysis

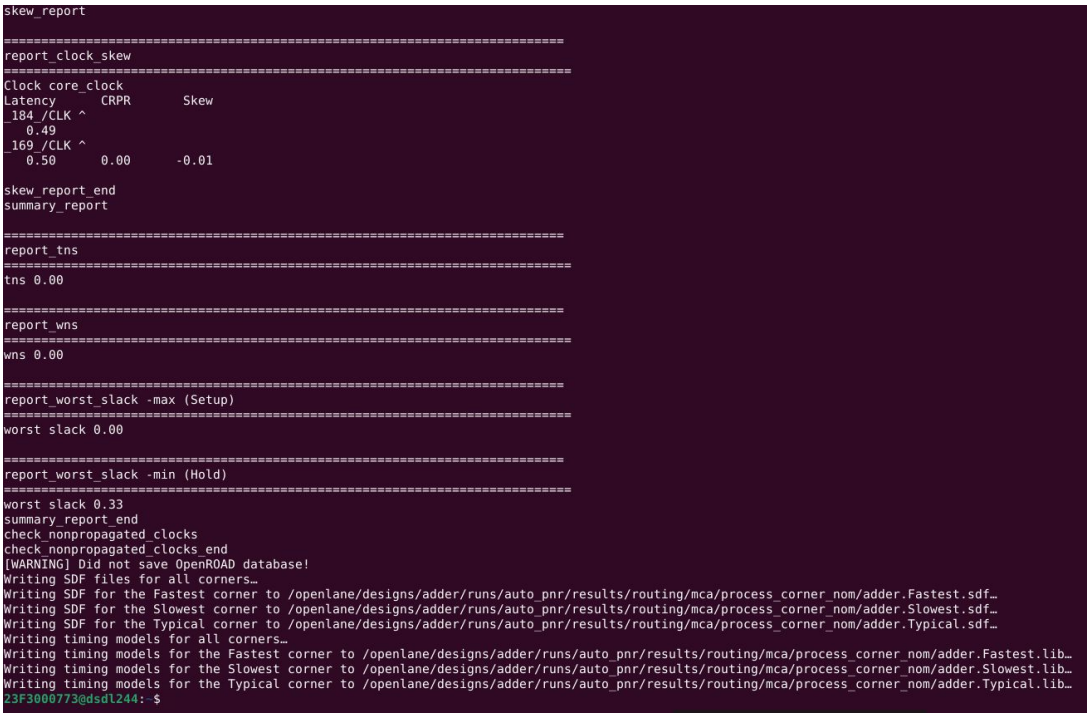

Figure 3: STA Report for Registered Ripple Carry Adder

Section 2: Registered Brent-Kung Adder

This section covers the registered version of the 16-bit Brent-Kung parallel prefix adder implementation.

2.1 Brent-Kung Adder with Registers

The Brent-Kung adder is a parallel prefix adder that balances speed and area complexity by minimizing the number of prefix nodes.

Brent-Kung Adder Verilog code

```
// sch_path: /home/global/23F3000773/tutorial_9/task3/brent_kung/bitBre
module brent
(
input [15:0] a_q, b_q,
    input cin_q, clk,
    output reg cout_q,
    output reg[15:0] sum_q);

    always@(posedge clk) begin
        A <= a_q;
        B <= b_q;
        Cin <= cin_q;
        sum_q <= S;
        cout_q <= Cout;
    end
wire net10 ;
wire net11 ;
wire net12 ;
wire net13 ;
wire net14 ;
wire net15 ;
wire net16 ;
wire net17 ;
wire net18 ;
wire net19 ;
wire net20 ;
wire net21 ;
wire net22 ;
wire net23 ;
wire net24 ;
wire net25 ;
wire net26 ;
wire net27 ;
wire net28 ;
wire net29 ;
wire net30 ;
```

```
wire net31 ;
wire net32 ;
wire net33 ;
wire net34 ;
wire net35 ;
wire net36 ;
wire net37 ;
wire net38 ;
wire net39 ;
wire net40 ;
wire net41 ;
wire net42 ;
wire net43 ;
wire net44 ;
wire net45 ;
wire net46 ;
wire net47 ;
wire net48 ;
wire net49 ;
wire net50 ;
wire net51 ;
wire net52 ;
wire net53 ;
wire net54 ;
wire net55 ;
wire net56 ;
wire net57 ;
wire net58 ;
wire net59 ;
wire net60 ;
wire net61 ;
wire net62 ;
wire net63 ;
wire net64 ;
wire net65 ;
wire net66 ;
wire net67 ;
wire net68 ;
wire net69 ;
wire net70 ;
wire net71 ;
wire net72 ;
wire net73 ;
wire net74 ;
wire net75 ;
wire net76 ;
wire net77 ;
wire net78 ;
```

```
wire net79 ;
wire net80 ;
wire net81 ;
wire net82 ;
wire net83 ;
wire net84 ;
wire net85 ;
wire net86 ;
wire net87 ;
wire net88 ;
wire net89 ;
wire net90 ;
wire net91 ;
logic [15:0] S ;
logic [15:0] A ;
logic Cout ;
logic [15:0] B ;
wire net1 ;
wire net2 ;
wire net3 ;
wire net4 ;
wire net5 ;
wire net6 ;
wire net7 ;
wire net8 ;
wire net9 ;
logic Cin ;

pg
x1 (
 .A( A[0] ),
 .G( net3 ),
 .P( net4 ),
 .B( B[0] )
);


pg
x2 (
 .A( A[1] ),
 .G( net2 ),
 .P( net1 ),
 .B( B[1] )
);


pg
x3 (
```

```
  .A( A[2] ),
  .G( net7 ),
  .P( net8 ),
  .B( B[2] )
);


pg
x4 (
  .A( A[3] ),
  .G( net6 ),
  .P( net5 ),
  .B( B[3] )
);


pg
x5 (
  .A( A[4] ),
  .G( net11 ),
  .P( net12 ),
  .B( B[4] )
);


pg
x6 (
  .A( A[5] ),
  .G( net10 ),
  .P( net9 ),
  .B( B[5] )
);


pg
x7 (
  .A( A[6] ),
  .G( net15 ),
  .P( net16 ),
  .B( B[6] )
);


pg
x8 (
  .A( A[7] ),
  .G( net14 ),
  .P( net13 ),
```

```
  .B( B[7] )
);


pg
x9 (
  .A( A[8] ),
  .G( net19 ),
  .P( net20 ),
  .B( B[8] )
);


pg
x10 (
  .A( A[9] ),
  .G( net18 ),
  .P( net17 ),
  .B( B[9] )
);


pg
x11 (
  .A( A[10] ),
  .G( net23 ),
  .P( net24 ),
  .B( B[10] )
);


pg
x12 (
  .A( A[11] ),
  .G( net22 ),
  .P( net21 ),
  .B( B[11] )
);


pg
x13 (
  .A( A[12] ),
  .G( net27 ),
  .P( net28 ),
  .B( B[12] )
);
```

```
pg
x14 (
 .A( A[13] ),
 .G( net26 ),
 .P( net25 ),
 .B( B[13] )
);


pg
x15 (
 .A( A[14] ),
 .G( net31 ),
 .P( net32 ),
 .B( B[14] )
);


pg
x16 (
 .A( A[15] ),
 .G( net30 ),
 .P( net29 ),
 .B( B[15] )
);


dot_op
x17 (
 .Pin( net1 ),
 .Pout( net46 ),
 .Gin( net2 ),
 .Gout( net45 ),
 .Pinprv( net4 ),
 .Ginprv( net3 )
);


dot_op
x18 (
 .Pin( net5 ),
 .Pout( net33 ),
 .Gin( net6 ),
 .Gout( net34 ),
 .Pinprv( net8 ),
 .Ginprv( net7 )
);
```

```
dot_op
x19 (
 .Pin( net9 ),
 .Pout( net48 ),
 .Gin( net10 ),
 .Gout( net47 ),
 .Pinprv( net12 ),
 .Ginprv( net11 )
);


dot_op
x20 (
 .Pin( net13 ),
 .Pout( net35 ),
 .Gin( net14 ),
 .Gout( net36 ),
 .Pinprv( net16 ),
 .Ginprv( net15 )
);


dot_op
x21 (
 .Pin( net17 ),
 .Pout( net50 ),
 .Gin( net18 ),
 .Gout( net49 ),
 .Pinprv( net20 ),
 .Ginprv( net19 )
);


dot_op
x22 (
 .Pin( net21 ),
 .Pout( net37 ),
 .Gin( net22 ),
 .Gout( net38 ),
 .Pinprv( net24 ),
 .Ginprv( net23 )
);


dot_op
x23 (
```

```
  .Pin( net25 ),
  .Pout( net52 ),
  .Gin( net26 ),
  .Gout( net51 ),
  .Pinprv( net28 ),
  .Ginprv( net27 )
);


dot_op
x24 (
  .Pin( net29 ),
  .Pout( net39 ),
  .Gin( net30 ),
  .Gout( net40 ),
  .Pinprv( net32 ),
  .Ginprv( net31 )
);


dot_op
x25 (
  .Pin( net33 ),
  .Pout( net54 ),
  .Gin( net34 ),
  .Gout( net53 ),
  .Pinprv( net46 ),
  .Ginprv( net45 )
);


dot_op
x26 (
  .Pin( net35 ),
  .Pout( net41 ),
  .Gin( net36 ),
  .Gout( net42 ),
  .Pinprv( net48 ),
  .Ginprv( net47 )
);


dot_op
x27 (
  .Pin( net37 ),
  .Pout( net56 ),
  .Gin( net38 ),
  .Gout( net55 ),
```

```
  .Pinprv( net50 ),
  .Ginprv( net49 )
);


dot_op
x28 (
  .Pin( net39 ),
  .Pout( net43 ),
  .Gin( net40 ),
  .Gout( net44 ),
  .Pinprv( net52 ),
  .Ginprv( net51 )
);


dot_op
x29 (
  .Pin( net41 ),
  .Pout( net58 ),
  .Gin( net42 ),
  .Gout( net57 ),
  .Pinprv( net54 ),
  .Ginprv( net53 )
);


dot_op
x30 (
  .Pin( net43 ),
  .Pout( net60 ),
  .Gin( net44 ),
  .Gout( net59 ),
  .Pinprv( net56 ),
  .Ginprv( net55 )
);


circle_op
x31 (
  .Pin( net4 ),
  .Gin( net3 ),
  .Cout( net75 ),
  .Pout( net80 ),
  .Cin( Cin )
);
```

```
circle_op
x32 (
 .Pin( net46 ),
 .Gin( net45 ),
 .Cout( net61 ),
 .Pout( net84 ),
 .Cin( Cin )
);


circle_op
x33 (
 .Pin( net8 ),
 .Gin( net7 ),
 .Cout( net74 ),
 .Pout( net83 ),
 .Cin( net61 )
);


circle_op
x34 (
 .Pin( net54 ),
 .Gin( net53 ),
 .Cout( net62 ),
 .Pout( net85 ),
 .Cin( Cin )
);


circle_op
x35 (
 .Pin( net12 ),
 .Gin( net11 ),
 .Cout( net73 ),
 .Pout( net82 ),
 .Cin( net62 )
);


circle_op
x36 (
 .Pin( net48 ),
 .Gin( net47 ),
 .Cout( net63 ),
 .Pout( net86 ),
 .Cin( net62 )
);
```

```
circle_op
x37 (
 .Pin( net16 ),
 .Gin( net15 ),
 .Cout( net72 ),
 .Pout( net79 ),
 .Cin( net63 )
);


circle_op
x38 (
 .Pin( net58 ),
 .Gin( net57 ),
 .Cout( net64 ),
 .Pout( net87 ),
 .Cin( Cin )
);


circle_op
x39 (
 .Pin( net20 ),
 .Gin( net19 ),
 .Cout( net71 ),
 .Pout( net81 ),
 .Cin( net64 )
);


circle_op
x40 (
 .Pin( net50 ),
 .Gin( net49 ),
 .Cout( net65 ),
 .Pout( net88 ),
 .Cin( net64 )
);


circle_op
x41 (
 .Pin( net24 ),
 .Gin( net23 ),
 .Cout( net70 ),
 .Pout( net78 ),
```

```
 .Cin( net65 )
);


circle_op
x42 (
 .Pin( net56 ),
 .Gin( net55 ),
 .Cout( net66 ),
 .Pout( net89 ),
 .Cin( net64 )
);


circle_op
x43 (
 .Pin( net60 ),
 .Gin( net59 ),
 .Cout( Cout ),
 .Pout( net90 ),
 .Cin( net64 )
);


circle_op
x44 (
 .Pin( net28 ),
 .Gin( net27 ),
 .Cout( net69 ),
 .Pout( net77 ),
 .Cin( net66 )
);


circle_op
x45 (
 .Pin( net52 ),
 .Gin( net51 ),
 .Cout( net67 ),
 .Pout( net91 ),
 .Cin( net66 )
);


circle_op
x46 (
 .Pin( net32 ),
 .Gin( net31 ),
```

```
  .Cout( net68 ),
  .Pout( net76 ),
  .Cin( net67 )
);

// noconn Cin
// noconn Cin
// noconn Cin
// noconn Cin

Sum
x47 (
  .Pin( net80 ),
  .Cin( Cin ),
  .Sum( S[0] )
);


Sum
x48 (
  .Pin( net1 ),
  .Cin( net75 ),
  .Sum( S[1] )
);


Sum
x49 (
  .Pin( net83 ),
  .Cin( net61 ),
  .Sum( S[2] )
);


Sum
x50 (
  .Pin( net5 ),
  .Cin( net74 ),
  .Sum( S[3] )
);


Sum
x51 (
  .Pin( net82 ),
  .Cin( net62 ),
  .Sum( S[4] )
);
```

```
Sum
x52 (
  .Pin( net9 ),
  .Cin( net73 ),
  .Sum( S[5] )
);


Sum
x53 (
  .Pin( net79 ),
  .Cin( net63 ),
  .Sum( S[6] )
);


Sum
x54 (
  .Pin( net13 ),
  .Cin( net72 ),
  .Sum( S[7] )
);


Sum
x55 (
  .Pin( net81 ),
  .Cin( net64 ),
  .Sum( S[8] )
);


Sum
x56 (
  .Pin( net17 ),
  .Cin( net71 ),
  .Sum( S[9] )
);


Sum
x57 (
  .Pin( net78 ),
  .Cin( net65 ),
  .Sum( S[10] )
);
```

```
Sum
x58 (
  .Pin( net21 ),
  .Cin( net70 ),
  .Sum( S[11] )
);


Sum
x59 (
  .Pin( net77 ),
  .Cin( net66 ),
  .Sum( S[12] )
);


Sum
x60 (
  .Pin( net25 ),
  .Cin( net69 ),
  .Sum( S[13] )
);


Sum
x61 (
  .Pin( net76 ),
  .Cin( net67 ),
  .Sum( S[14] )
);


Sum
x62 (
  .Pin( net29 ),
  .Cin( net68 ),
  .Sum( S[15] )
);

// noconn Cin
// noconn Cout
// noconn S[15]
// noconn S[14]
// noconn S[13]
// noconn S[12]
// noconn S[11]
```

```
// noconn S[10]
// noconn S[9]
// noconn S[8]
// noconn S[7]
// noconn S[6]
// noconn S[5]
// noconn S[4]
// noconn S[3]
// noconn S[2]
// noconn S[1]
// noconn S[0]
// noconn A[15]
// noconn A[14]
// noconn A[13]
// noconn A[12]
// noconn A[11]
// noconn A[10]
// noconn A[9]
// noconn A[8]
// noconn A[7]
// noconn A[6]
// noconn A[5]
// noconn A[4]
// noconn A[3]
// noconn A[2]
// noconn A[1]
// noconn A[0]
// noconn B[15]
// noconn B[14]
// noconn B[13]
// noconn B[12]
// noconn B[11]
// noconn B[10]
// noconn B[9]
// noconn B[8]
// noconn B[7]
// noconn B[6]
// noconn B[5]
// noconn B[4]
// noconn B[3]
// noconn B[2]
// noconn B[1]
// noconn B[0]
```

endmodule

```
// expanding   symbol:  pg.sym # of pins=4
// sym_path: /home/global/23F3000773/tutorial_9/task3/brent_kung/pg.sym
```

```
// sch_path: /home/global/23F3000773/tutorial_9/task3/brent_kung/pg.sch
module pg
(
  input logic A,
  output logic G,
  output logic P,
  input logic B
);
assign G = A & B;
assign P = A ^ B;

// noconn A
// noconn B
// noconn G
// noconn P
endmodule

// expanding   symbol:  dot_op.sym # of pins=6
// sym_path: /home/global/23F3000773/tutorial_9/task3/brent_kung/dot_op
// sch_path: /home/global/23F3000773/tutorial_9/task3/brent_kung/dot_op
module dot_op
(
  input logic Pin,
  output logic Pout,
  input logic Gin,
  output logic Gout,
  input logic Pinprv,
  input logic Ginprv
);
assign Pout = Pin & Pinprv;
assign Gout = Gin | (Pin & Ginprv);
// noconn Pin
// noconn Gin
// noconn Pinprv
// noconn Pout
// noconn Gout
// noconn Ginprv
endmodule

// expanding   symbol:  circle_op.sym # of pins=5
// sym_path: /home/global/23F3000773/tutorial_9/task3/brent_kung/circle
// sch_path: /home/global/23F3000773/tutorial_9/task3/brent_kung/circle
module circle_op
(
  input logic Pin,
  input logic Gin,
  output logic Cout,
  output logic Pout,
```

```
    input logic Cin
  );
  assign Pout = Pin;
  assign Cout = Gin | (Pin & Cin);
  // noconn Pin
  // noconn Gin
  // noconn Cout
  // noconn Cin
  // noconn Pout
  endmodule

  // expanding   symbol:  Sum.sym # of pins=3
  // sym_path: /home/global/23F3000773/tutorial_9/task3/brent_kung/Sum.sy
  // sch_path: /home/global/23F3000773/tutorial_9/task3/brent_kung/Sum.sc
  module Sum
  (
    input logic Pin,
    input logic Cin,
    output logic Sum
  );
  assign Sum = Pin ^ Cin;

  // noconn Pin
  // noconn Cin
  // noconn Sum
endmodule
```

OpenLane Layout

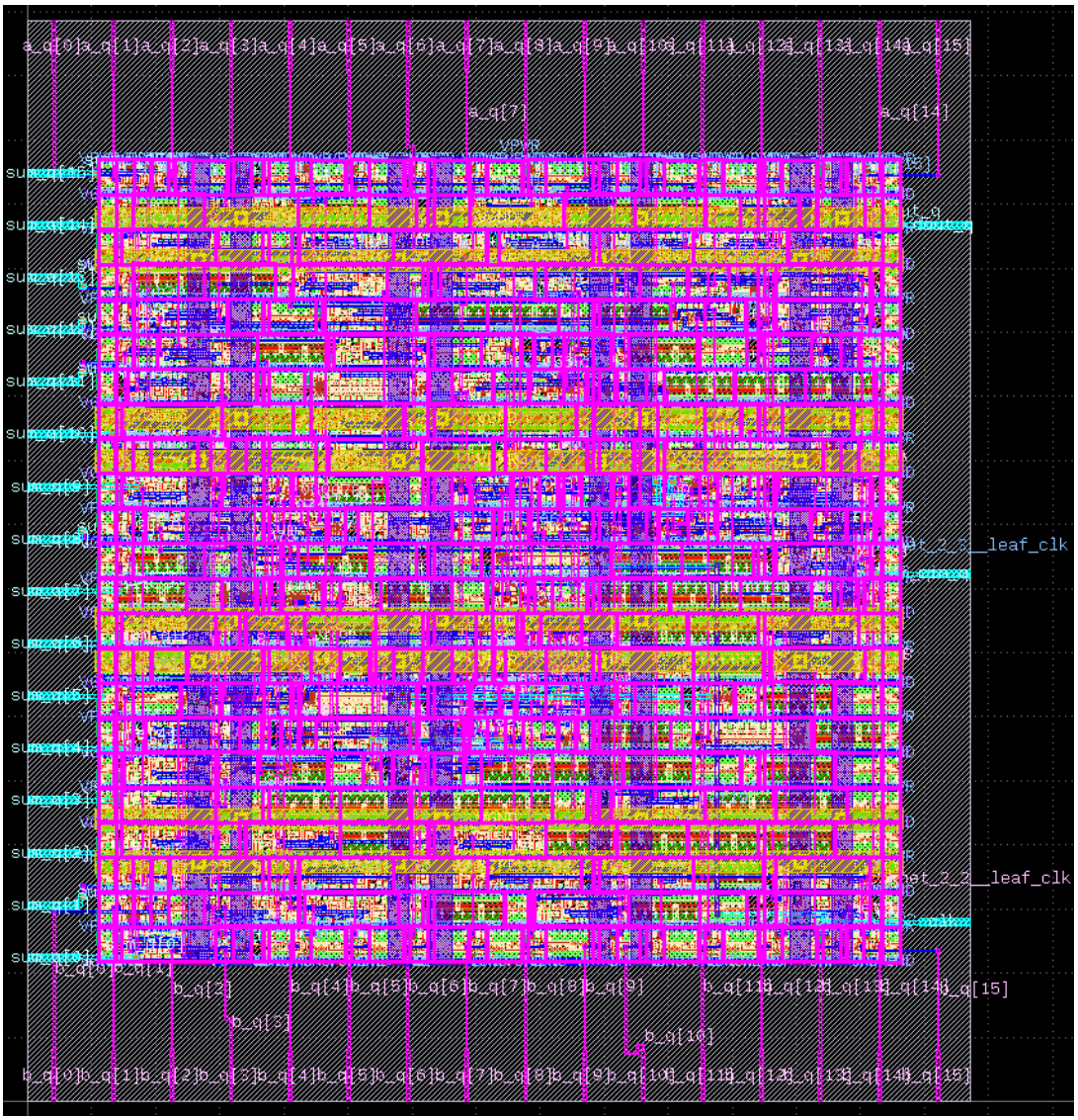

Figure 4: OpenLane Layout for Registered Brent-Kung Adder

Static Timing Analysis

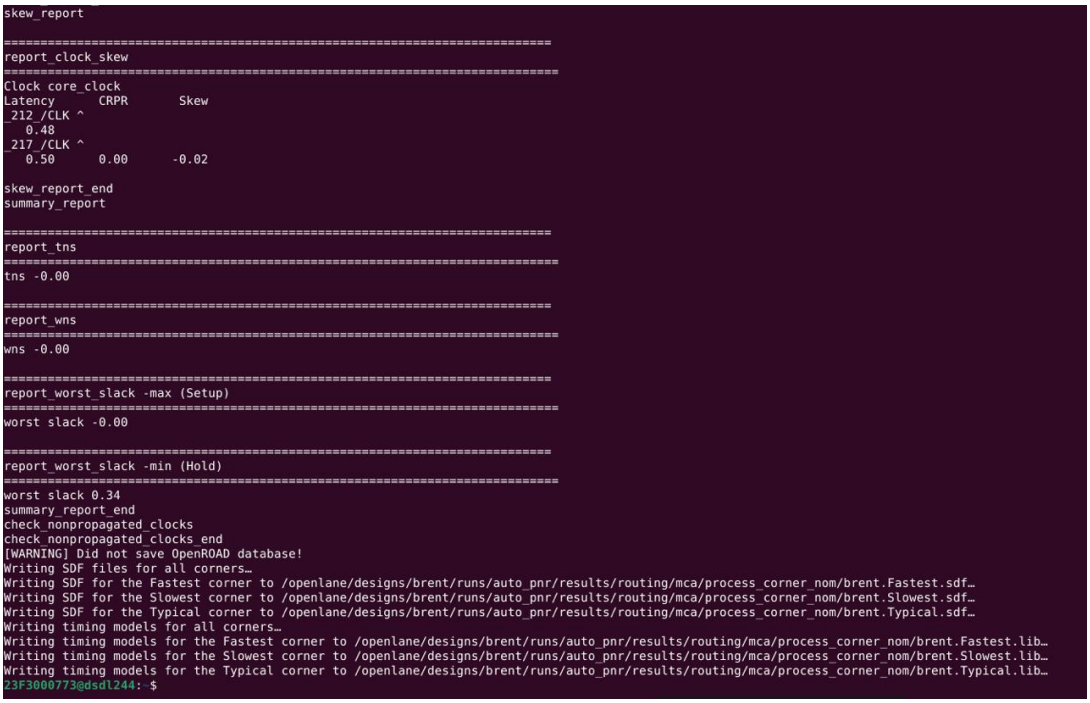

Figure 5: STA Report for Registered Brent-Kung Adder

Section 3: Registered Kogge-Stone Adder

This section covers the registered version of the 16-bit Kogge-Stone parallel prefix adder implementation.

3.1 Kogge-Stone Adder with Registers

The Kogge-Stone adder is a parallel prefix adder that achieves minimal depth at the cost of higher area and wiring complexity.

Kogge-Stone Adder Verilog Code

```
// sch_path: /home/global/23F3000773/tutorial_9/task3/kogge_stone/bitK
module kogge(input [15:0] a_q, b_q,
    input cin_q, clk,
    output reg cout_q,
    output reg[15:0] sum_q);

    always@(posedge clk) begin
        A <= a_q;
        B <= b_q;
        Cin <= cin_q;
        sum_q <= S;
        cout_q <= Cout;
    end
wire net10 ;
wire net11 ;
wire net12 ;
wire net13 ;
wire net14 ;
wire net15 ;
wire net16 ;
wire net17 ;
wire net18 ;
wire net19 ;
wire net20 ;
wire net21 ;
wire net22 ;
wire net23 ;
wire net24 ;
wire net25 ;
wire net26 ;
wire net27 ;
wire net28 ;
wire net29 ;
wire net30 ;
wire net31 ;
wire net32 ;
```

```
wire net33 ;
wire net34 ;
wire net35 ;
wire net36 ;
wire net37 ;
wire net38 ;
wire net39 ;
wire net40 ;
wire net41 ;
wire net42 ;
wire net43 ;
wire net44 ;
wire net45 ;
wire net46 ;
wire net47 ;
wire net48 ;
wire net49 ;
wire net50 ;
wire net51 ;
wire net52 ;
wire net53 ;
wire net54 ;
wire net55 ;
wire net56 ;
wire net57 ;
wire net58 ;
wire net59 ;
wire net60 ;
wire net61 ;
wire net62 ;
wire net63 ;
wire net64 ;
wire net65 ;
wire net66 ;
wire net67 ;
wire net68 ;
wire net69 ;
wire net70 ;
wire net71 ;
wire net72 ;
wire net73 ;
wire net74 ;
wire net75 ;
wire net76 ;
wire net77 ;
wire net78 ;
wire net79 ;
wire net80 ;
```

```
wire net81 ;
wire net82 ;
wire net83 ;
wire net84 ;
wire net85 ;
wire net86 ;
wire net87 ;
wire net88 ;
wire net89 ;
wire net90 ;
wire net91 ;
wire net92 ;
wire net93 ;
wire net94 ;
wire net95 ;
wire net96 ;
wire net97 ;
wire net98 ;
wire net99 ;
logic [15:0] S ;
logic [15:0] A ;
logic Cout ;
logic [15:0] B ;
wire net1 ;
wire net2 ;
wire net3 ;
wire net4 ;
wire net5 ;
wire net6 ;
wire net7 ;
wire net8 ;
wire net9 ;
logic Cin ;
wire net100 ;
wire net101 ;
wire net102 ;
wire net103 ;
wire net104 ;
wire net105 ;
wire net106 ;
wire net107 ;
wire net108 ;
wire net109 ;
wire net110 ;
wire net111 ;
wire net112 ;
wire net113 ;
wire net114 ;
```

```
wire net115 ;
wire net116 ;
wire net117 ;
wire net118 ;
wire net119 ;
wire net120 ;
wire net121 ;
wire net122 ;
wire net123 ;
wire net124 ;
wire net125 ;
wire net126 ;
wire net127 ;
wire net128 ;
wire net129 ;
wire net130 ;
wire net131 ;
wire net132 ;
wire net133 ;
wire net134 ;
wire net135 ;
wire net136 ;
wire net137 ;
wire net138 ;
wire net139 ;
wire net140 ;
wire net141 ;
wire net142 ;
wire net143 ;
wire net144 ;
wire net145 ;
wire net146 ;
wire net147 ;
wire net148 ;
wire net149 ;
wire net150 ;
wire net151 ;
wire net152 ;
wire net153 ;
wire net154 ;
wire net155 ;
wire net156 ;
wire net157 ;
wire net158 ;
wire net159 ;
wire net160 ;
wire net161 ;
```

```
pg
x1 (
  .A( A[0] ),
  .G( net25 ),
  .P( net24 ),
  .B( B[0] )
);


pg
x2 (
  .A( A[1] ),
  .G( net23 ),
  .P( net22 ),
  .B( B[1] )
);


pg
x3 (
  .A( A[2] ),
  .G( net28 ),
  .P( net27 ),
  .B( B[2] )
);


pg
x4 (
  .A( A[3] ),
  .G( net26 ),
  .P( net21 ),
  .B( B[3] )
);


pg
x5 (
  .A( A[4] ),
  .G( net31 ),
  .P( net30 ),
  .B( B[4] )
);


pg
x6 (
  .A( A[5] ),
```

```
  .G( net29 ),
  .P( net20 ),
  .B( B[5] )
);


pg
x7 (
  .A( A[6] ),
  .G( net34 ),
  .P( net33 ),
  .B( B[6] )
);


pg
x8 (
  .A( A[7] ),
  .G( net32 ),
  .P( net19 ),
  .B( B[7] )
);


pg
x9 (
  .A( A[8] ),
  .G( net128 ),
  .P( net125 ),
  .B( B[8] )
);


pg
x10 (
  .A( A[9] ),
  .G( net37 ),
  .P( net18 ),
  .B( B[9] )
);


pg
x11 (
  .A( A[10] ),
  .G( net36 ),
  .P( net35 ),
  .B( B[10] )
```

```
);


pg
x12 (
  .A( A[11] ),
  .G( net40 ),
  .P( net17 ),
  .B( B[11] )
);


pg
x13 (
  .A( A[12] ),
  .G( net39 ),
  .P( net38 ),
  .B( B[12] )
);


pg
x14 (
  .A( A[13] ),
  .G( net43 ),
  .P( net16 ),
  .B( B[13] )
);


pg
x15 (
  .A( A[14] ),
  .G( net42 ),
  .P( net41 ),
  .B( B[14] )
);


pg
x16 (
  .A( A[15] ),
  .G( net129 ),
  .P( net124 ),
  .B( B[15] )
);
```

```
Sum
x47 (
  .Pin( net24 ),
  .Cin( Cin ),
  .Sum( S[0] )
);


Sum
x48 (
  .Pin( net22 ),
  .Cin( net2 ),
  .Sum( S[1] )
);


Sum
x49 (
  .Pin( net27 ),
  .Cin( net1 ),
  .Sum( S[2] )
);


Sum
x50 (
  .Pin( net21 ),
  .Cin( net3 ),
  .Sum( S[3] )
);


Sum
x51 (
  .Pin( net30 ),
  .Cin( net6 ),
  .Sum( S[4] )
);


Sum
x52 (
  .Pin( net20 ),
  .Cin( net5 ),
  .Sum( S[5] )
);
```

```
Sum
x53 (
  .Pin( net33 ),
  .Cin( net4 ),
  .Sum( S[6] )
);


Sum
x54 (
  .Pin( net19 ),
  .Cin( net7 ),
  .Sum( S[7] )
);


Sum
x55 (
  .Pin( net125 ),
  .Cin( net10 ),
  .Sum( S[8] )
);


Sum
x56 (
  .Pin( net18 ),
  .Cin( net9 ),
  .Sum( S[9] )
);


Sum
x57 (
  .Pin( net35 ),
  .Cin( net8 ),
  .Sum( S[10] )
);


Sum
x58 (
  .Pin( net17 ),
  .Cin( net11 ),
  .Sum( S[11] )
);
```

```
Sum
x59 (
  .Pin( net38 ),
  .Cin( net14 ),
  .Sum( S[12] )
);


Sum
x60 (
  .Pin( net16 ),
  .Cin( net13 ),
  .Sum( S[13] )
);


Sum
x61 (
  .Pin( net41 ),
  .Cin( net12 ),
  .Sum( S[14] )
);


Sum
x62 (
  .Pin( net124 ),
  .Cin( net15 ),
  .Sum( S[15] )
);

// noconn Cin
// noconn Cout
// noconn S[15]
// noconn S[14]
// noconn S[13]
// noconn S[12]
// noconn S[11]
// noconn S[10]
// noconn S[9]
// noconn S[8]
// noconn S[7]
// noconn S[6]
// noconn S[5]
// noconn S[4]
// noconn S[3]
// noconn S[2]
// noconn S[1]
```

```
// noconn S[0]
// noconn A[15]
// noconn A[14]
// noconn A[13]
// noconn A[12]
// noconn A[11]
// noconn A[10]
// noconn A[9]
// noconn A[8]
// noconn A[7]
// noconn A[6]
// noconn A[5]
// noconn A[4]
// noconn A[3]
// noconn A[2]
// noconn A[1]
// noconn A[0]
// noconn B[15]
// noconn B[14]
// noconn B[13]
// noconn B[12]
// noconn B[11]
// noconn B[10]
// noconn B[9]
// noconn B[8]
// noconn B[7]
// noconn B[6]
// noconn B[5]
// noconn B[4]
// noconn B[3]
// noconn B[2]
// noconn B[1]
// noconn B[0]


dot_op
x64 (
  .Pin( net124 ),
  .Pout( net98 ),
  .Gin( net129 ),
  .Gout( net99 ),
  .Pinprv( net41 ),
  .Ginprv( net42 )
);


dot_op
x63 (
```

```
  .Pin( net41 ),
  .Pout( net100 ),
  .Gin( net42 ),
  .Gout( net101 ),
  .Pinprv( net16 ),
  .Ginprv( net43 )
);


dot_op
x65 (
  .Pin( net16 ),
  .Pout( net67 ),
  .Gin( net43 ),
  .Gout( net66 ),
  .Pinprv( net38 ),
  .Ginprv( net39 )
);


dot_op
x66 (
  .Pin( net38 ),
  .Pout( net69 ),
  .Gin( net39 ),
  .Gout( net68 ),
  .Pinprv( net17 ),
  .Ginprv( net40 )
);


dot_op
x67 (
  .Pin( net17 ),
  .Pout( net63 ),
  .Gin( net40 ),
  .Gout( net62 ),
  .Pinprv( net35 ),
  .Ginprv( net36 )
);


dot_op
x68 (
  .Pin( net35 ),
  .Pout( net65 ),
  .Gin( net36 ),
  .Gout( net64 ),
```

```
 .Pinprv( net18 ),
 .Ginprv( net37 )
);


dot_op
x69 (
 .Pin( net18 ),
 .Pout( net59 ),
 .Gin( net37 ),
 .Gout( net58 ),
 .Pinprv( net125 ),
 .Ginprv( net128 )
);


dot_op
x70 (
 .Pin( net125 ),
 .Pout( net61 ),
 .Gin( net128 ),
 .Gout( net60 ),
 .Pinprv( net19 ),
 .Ginprv( net32 )
);


dot_op
x71 (
 .Pin( net19 ),
 .Pout( net55 ),
 .Gin( net32 ),
 .Gout( net54 ),
 .Pinprv( net33 ),
 .Ginprv( net34 )
);


dot_op
x72 (
 .Pin( net33 ),
 .Pout( net57 ),
 .Gin( net34 ),
 .Gout( net56 ),
 .Pinprv( net20 ),
 .Ginprv( net29 )
);
```

```
dot_op
x73 (
 .Pin( net20 ),
 .Pout( net51 ),
 .Gin( net29 ),
 .Gout( net50 ),
 .Pinprv( net30 ),
 .Ginprv( net31 )
);


dot_op
x74 (
 .Pin( net30 ),
 .Pout( net53 ),
 .Gin( net31 ),
 .Gout( net52 ),
 .Pinprv( net21 ),
 .Ginprv( net26 )
);


dot_op
x75 (
 .Pin( net21 ),
 .Pout( net47 ),
 .Gin( net26 ),
 .Gout( net46 ),
 .Pinprv( net27 ),
 .Ginprv( net28 )
);


dot_op
x76 (
 .Pin( net27 ),
 .Pout( net49 ),
 .Gin( net28 ),
 .Gout( net48 ),
 .Pinprv( net22 ),
 .Ginprv( net23 )
);


dot_op
x77 (
 .Pin( net22 ),
```

```
  .Pout( net45 ),
  .Gin( net23 ),
  .Gout( net44 ),
  .Pinprv( net24 ),
  .Ginprv( net25 )
);


dot_op
x78 (
  .Pin( net98 ),
  .Pout( net70 ),
  .Gin( net99 ),
  .Gout( net71 ),
  .Pinprv( net67 ),
  .Ginprv( net66 )
);


dot_op
x79 (
  .Pin( net100 ),
  .Pout( net72 ),
  .Gin( net101 ),
  .Gout( net73 ),
  .Pinprv( net69 ),
  .Ginprv( net68 )
);


dot_op
x80 (
  .Pin( net67 ),
  .Pout( net74 ),
  .Gin( net66 ),
  .Gout( net75 ),
  .Pinprv( net63 ),
  .Ginprv( net62 )
);


dot_op
x81 (
  .Pin( net69 ),
  .Pout( net76 ),
  .Gin( net68 ),
  .Gout( net77 ),
  .Pinprv( net65 ),
```

```
  .Ginprv( net64 )
);


dot_op
x82 (
  .Pin( net63 ),
  .Pout( net78 ),
  .Gin( net62 ),
  .Gout( net79 ),
  .Pinprv( net59 ),
  .Ginprv( net58 )
);


dot_op
x83 (
  .Pin( net65 ),
  .Pout( net80 ),
  .Gin( net64 ),
  .Gout( net81 ),
  .Pinprv( net61 ),
  .Ginprv( net60 )
);


dot_op
x84 (
  .Pin( net59 ),
  .Pout( net82 ),
  .Gin( net58 ),
  .Gout( net83 ),
  .Pinprv( net55 ),
  .Ginprv( net54 )
);


dot_op
x85 (
  .Pin( net61 ),
  .Pout( net84 ),
  .Gin( net60 ),
  .Gout( net85 ),
  .Pinprv( net57 ),
  .Ginprv( net56 )
);
```

```
dot_op
x86 (
 .Pin( net55 ),
 .Pout( net86 ),
 .Gin( net54 ),
 .Gout( net87 ),
 .Pinprv( net51 ),
 .Ginprv( net50 )
);


dot_op
x87 (
 .Pin( net57 ),
 .Pout( net88 ),
 .Gin( net56 ),
 .Gout( net89 ),
 .Pinprv( net53 ),
 .Ginprv( net52 )
);


dot_op
x88 (
 .Pin( net51 ),
 .Pout( net90 ),
 .Gin( net50 ),
 .Gout( net91 ),
 .Pinprv( net47 ),
 .Ginprv( net46 )
);


dot_op
x89 (
 .Pin( net53 ),
 .Pout( net92 ),
 .Gin( net52 ),
 .Gout( net93 ),
 .Pinprv( net49 ),
 .Ginprv( net48 )
);


dot_op
x90 (
 .Pin( net47 ),
 .Pout( net95 ),
```

```
 .Gin( net46 ),
 .Gout( net94 ),
 .Pinprv( net45 ),
 .Ginprv( net44 )
);


dot_op
x91 (
 .Pin( net49 ),
 .Pout( net97 ),
 .Gin( net48 ),
 .Gout( net96 ),
 .Pinprv( net24 ),
 .Ginprv( net25 )
);


dot_op
x93 (
 .Pin( net70 ),
 .Pout( net104 ),
 .Gin( net71 ),
 .Gout( net105 ),
 .Pinprv( net78 ),
 .Ginprv( net79 )
);


dot_op
x94 (
 .Pin( net72 ),
 .Pout( net106 ),
 .Gin( net73 ),
 .Gout( net107 ),
 .Pinprv( net80 ),
 .Ginprv( net81 )
);


dot_op
x95 (
 .Pin( net74 ),
 .Pout( net108 ),
 .Gin( net75 ),
 .Gout( net109 ),
 .Pinprv( net82 ),
 .Ginprv( net83 )
```

```
);


dot_op
x96 (
 .Pin( net76 ),
 .Pout( net110 ),
 .Gin( net77 ),
 .Gout( net111 ),
 .Pinprv( net84 ),
 .Ginprv( net85 )
);


dot_op
x97 (
 .Pin( net78 ),
 .Pout( net112 ),
 .Gin( net79 ),
 .Gout( net113 ),
 .Pinprv( net86 ),
 .Ginprv( net87 )
);


dot_op
x98 (
 .Pin( net80 ),
 .Pout( net114 ),
 .Gin( net81 ),
 .Gout( net126 ),
 .Pinprv( net88 ),
 .Ginprv( net89 )
);


dot_op
x99 (
 .Pin( net82 ),
 .Pout( net127 ),
 .Gin( net83 ),
 .Gout( net115 ),
 .Pinprv( net90 ),
 .Ginprv( net91 )
);


dot_op
```

```
x100 (
 .Pin( net84 ),
 .Pout( net116 ),
 .Gin( net85 ),
 .Gout( net117 ),
 .Pinprv( net92 ),
 .Ginprv( net93 )
);


dot_op
x101 (
 .Pin( net86 ),
 .Pout( net103 ),
 .Gin( net87 ),
 .Gout( net102 ),
 .Pinprv( net95 ),
 .Ginprv( net94 )
);


dot_op
x102 (
 .Pin( net88 ),
 .Pout( net118 ),
 .Gin( net89 ),
 .Gout( net119 ),
 .Pinprv( net97 ),
 .Ginprv( net96 )
);


dot_op
x103 (
 .Pin( net90 ),
 .Pout( net120 ),
 .Gin( net91 ),
 .Gout( net121 ),
 .Pinprv( net45 ),
 .Ginprv( net44 )
);


dot_op
x104 (
 .Pin( net92 ),
 .Pout( net122 ),
 .Gin( net93 ),
```

```
 .Gout( net123 ),
 .Pinprv( net24 ),
 .Ginprv( net25 )
);


dot_op
x92 (
 .Pin( net104 ),
 .Pout( net130 ),
 .Gin( net105 ),
 .Gout( net131 ),
 .Pinprv( net103 ),
 .Ginprv( net102 )
);


dot_op
x105 (
 .Pin( net106 ),
 .Pout( net132 ),
 .Gin( net107 ),
 .Gout( net133 ),
 .Pinprv( net118 ),
 .Ginprv( net119 )
);


dot_op
x106 (
 .Pin( net108 ),
 .Pout( net134 ),
 .Gin( net109 ),
 .Gout( net135 ),
 .Pinprv( net120 ),
 .Ginprv( net121 )
);


dot_op
x107 (
 .Pin( net110 ),
 .Pout( net136 ),
 .Gin( net111 ),
 .Gout( net137 ),
 .Pinprv( net122 ),
 .Ginprv( net123 )
);
```

```
dot_op
x108 (
 .Pin( net112 ),
 .Pout( net138 ),
 .Gin( net113 ),
 .Gout( net139 ),
 .Pinprv( net95 ),
 .Ginprv( net94 )
);


dot_op
x109 (
 .Pin( net114 ),
 .Pout( net140 ),
 .Gin( net126 ),
 .Gout( net141 ),
 .Pinprv( net97 ),
 .Ginprv( net96 )
);


dot_op
x110 (
 .Pin( net127 ),
 .Pout( net142 ),
 .Gin( net115 ),
 .Gout( net143 ),
 .Pinprv( net45 ),
 .Ginprv( net44 )
);


dot_op
x111 (
 .Pin( net116 ),
 .Pout( net144 ),
 .Gin( net117 ),
 .Gout( net145 ),
 .Pinprv( net24 ),
 .Ginprv( net25 )
);


circle_op
x17 (
```

```
 .Pin( net122 ),
 .Gin( net123 ),
 .Cout( net5 ),
 .Pout( net146 ),
 .Cin( Cin )
);

// noconn Cin

circle_op
x18 (
 .Pin( net95 ),
 .Gin( net94 ),
 .Cout( net6 ),
 .Pout( net147 ),
 .Cin( Cin )
);

// noconn Cin

circle_op
x19 (
 .Pin( net97 ),
 .Gin( net96 ),
 .Cout( net3 ),
 .Pout( net148 ),
 .Cin( Cin )
);

// noconn Cin

circle_op
x20 (
 .Pin( net45 ),
 .Gin( net44 ),
 .Cout( net1 ),
 .Pout( net149 ),
 .Cin( Cin )
);

// noconn Cin

circle_op
x21 (
 .Pin( net24 ),
 .Gin( net25 ),
 .Cout( net2 ),
 .Pout( net150 ),
```

```
  .Cin( Cin )
);

// noconn Cin

circle_op
x22 (
  .Pin( net142 ),
  .Gin( net143 ),
  .Cout( net8 ),
  .Pout( net151 ),
  .Cin( Cin )
);

// noconn Cin

circle_op
x23 (
  .Pin( net144 ),
  .Gin( net145 ),
  .Cout( net9 ),
  .Pout( net152 ),
  .Cin( Cin )
);

// noconn Cin

circle_op
x24 (
  .Pin( net103 ),
  .Gin( net102 ),
  .Cout( net10 ),
  .Pout( net153 ),
  .Cin( Cin )
);

// noconn Cin

circle_op
x25 (
  .Pin( net118 ),
  .Gin( net119 ),
  .Cout( net7 ),
  .Pout( net154 ),
  .Cin( Cin )
);

// noconn Cin
```

```
circle_op
x26 (
 .Pin( net120 ),
 .Gin( net121 ),
 .Cout( net4 ),
 .Pout( net155 ),
 .Cin( Cin )
);

// noconn Cin

circle_op
x27 (
 .Pin( net132 ),
 .Gin( net133 ),
 .Cout( net15 ),
 .Pout( net156 ),
 .Cin( Cin )
);

// noconn Cin

circle_op
x28 (
 .Pin( net134 ),
 .Gin( net135 ),
 .Cout( net12 ),
 .Pout( net157 ),
 .Cin( Cin )
);

// noconn Cin

circle_op
x29 (
 .Pin( net136 ),
 .Gin( net137 ),
 .Cout( net13 ),
 .Pout( net158 ),
 .Cin( Cin )
);

// noconn Cin

circle_op
x30 (
 .Pin( net138 ),
```

```
  .Gin( net139 ),
  .Cout( net14 ),
  .Pout( net159 ),
  .Cin( Cin )
);

// noconn Cin

circle_op
x31 (
  .Pin( net140 ),
  .Gin( net141 ),
  .Cout( net11 ),
  .Pout( net160 ),
  .Cin( Cin )
);

// noconn Cin

circle_op
x32 (
  .Pin( net130 ),
  .Gin( net131 ),
  .Cout( Cout ),
  .Pout( net161 ),
  .Cin( Cin )
);

// noconn Cin
endmodule

// expanding   symbol:  pg.sym # of pins=4
// sym_path: /home/global/23F3000773/tutorial_9/task3/kogge_stone/pg.sy
// sch_path: /home/global/23F3000773/tutorial_9/task3/kogge_stone/pg.sc
module pg
(
  input logic A,
  output logic G,
  output logic P,
  input logic B
);
assign G = A & B;
assign P = A ^ B;

// noconn A
// noconn B
// noconn G
// noconn P
```

```
endmodule

// expanding   symbol:  Sum.sym # of pins=3
// sym_path: /home/global/23F3000773/tutorial_9/task3/kogge_stone/Sum.s
// sch_path: /home/global/23F3000773/tutorial_9/task3/kogge_stone/Sum.s
module Sum
(
  input logic Pin,
  input logic Cin,
  output logic Sum
);
assign Sum = Pin ^ Cin;

// noconn Pin
// noconn Cin
// noconn Sum
endmodule

// expanding   symbol:  dot_op.sym # of pins=6
// sym_path: /home/global/23F3000773/tutorial_9/task3/kogge_stone/dot_o
// sch_path: /home/global/23F3000773/tutorial_9/task3/kogge_stone/dot_o
module dot_op
(
  input logic Pin,
  output logic Pout,
  input logic Gin,
  output logic Gout,
  input logic Pinprv,
  input logic Ginprv
);
assign Pout = Pin & Pinprv;
assign Gout = Gin | (Pin & Ginprv);
// noconn Pin
// noconn Gin
// noconn Pinprv
// noconn Pout
// noconn Gout
// noconn Ginprv
endmodule

// expanding   symbol:  circle_op.sym # of pins=5
// sym_path: /home/global/23F3000773/tutorial_9/task3/kogge_stone/circl
// sch_path: /home/global/23F3000773/tutorial_9/task3/kogge_stone/circl
module circle_op
(
  input logic Pin,
  input logic Gin,
  output logic Cout,
```

```
    output logic Pout,
    input logic Cin
);
assign Pout = Pin;
assign Cout = Gin | (Pin & Cin);
// noconn Pin
// noconn Gin
// noconn Cout
// noconn Cin
// noconn Pout
endmodule
```

OpenLane Layout

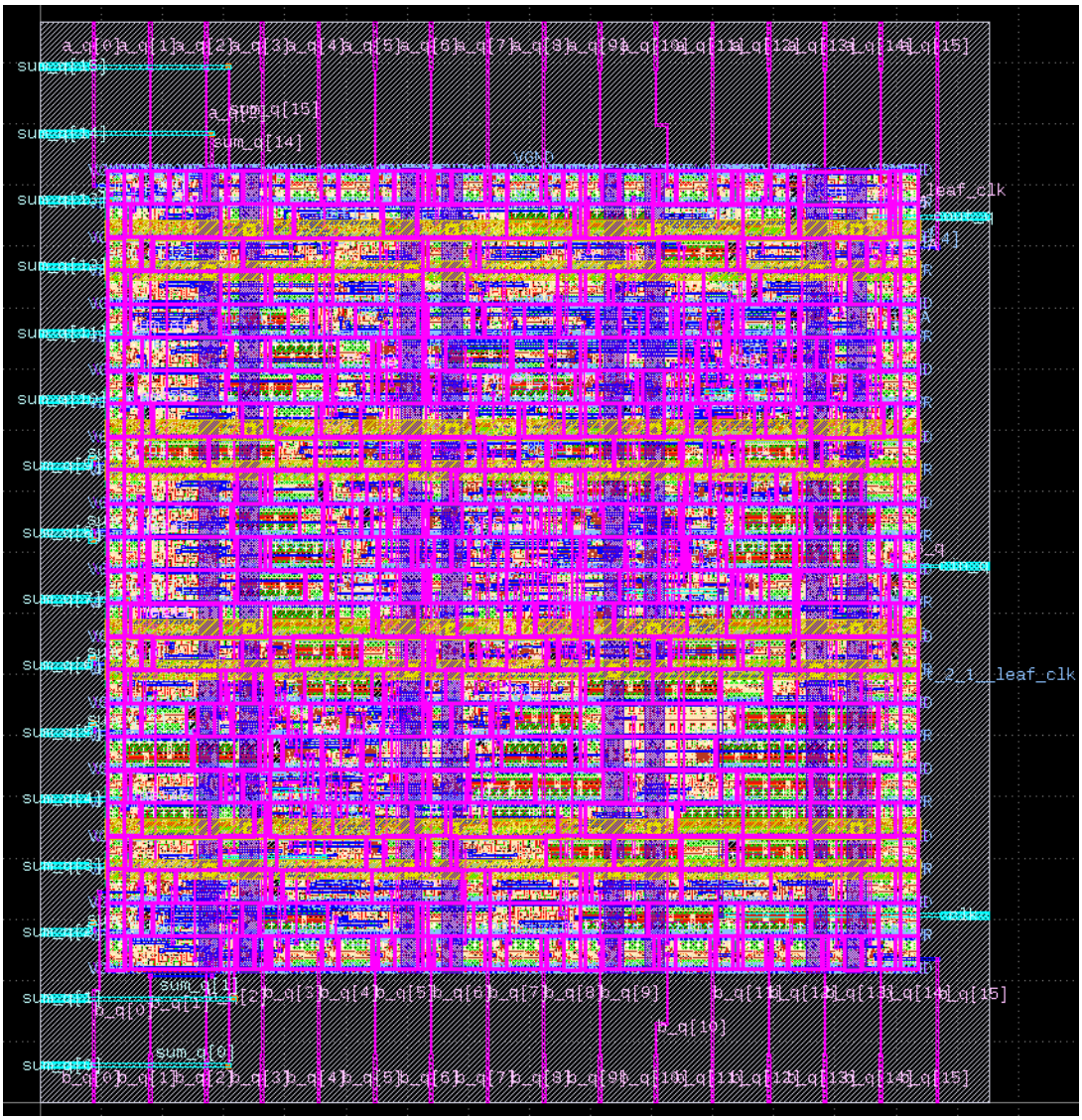

Figure 6: OpenLane Layout for Registered Kogge-Stone Adder

Static Timing Analysis

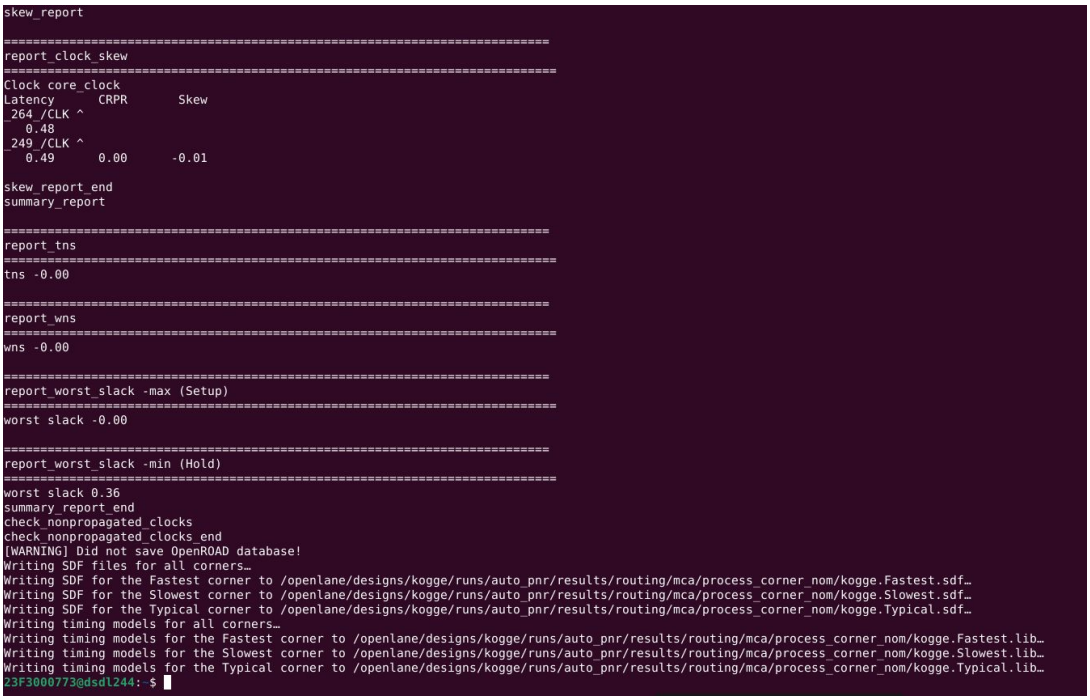

Figure 7: STA Report for Registered Kogge-Stone Adder

Section 4: Frequency Calculation and Performance Comparison

4.1 Maximum Operating Frequency

The maximum operating frequency for each adder is calculated from the critical path delay obtained from STA:

$$f_{max} = \frac{1}{T_{critical\_path}}$$

where $T_{critical\_path}$ is the worst-case propagation delay from input registers through the combinational logic to output registers.

Table 1: Performance Metrics for Registered Adders

Adder Type	**Max Frequency (MHz)**
Ripple Carry Adder	219.78
Brent-Kung Adder	207.03
Kogge-Stone Adder	247.52

Conclusion

This tutorial successfully implemented registered versions of three different adder architectures and analyzed their performance using the OpenLane physical design flow. The layouts were generated and static timing analysis was performed to determine the maximum operating frequencies.

The results demonstrate the trade-offs between different adder architectures:

- **Ripple Carry Adder**: Simple design with minimal area but slower critical path due to carry propagation
- **Brent-Kung Adder**: Balanced approach with moderate speed and area
- **Kogge-Stone Adder**: Fastest critical path with parallel prefix tree but higher area and routing complexity

The addition of input and output registers enables synchronous operation and facilitates accurate timing analysis in a clocked system, which is essential for real-world digital design implementations.